

✓ Subsets

Question

Given an integer array `nums` of unique elements, return all possible subsets (the power set).
The solution set must not contain duplicate subsets. Return the solution in any order.

Constraints

- $1 \leq \text{nums.length} \leq 10$
 - $-10 \leq \text{nums}[i] \leq 10$
 - All the numbers of `nums` are unique.
-

Solution

you should know how recursion works and ArrayList concept in java before solving this question. for doing this question firstly we understand two concepts --

✓ • pick and not pick pattern in recursion.

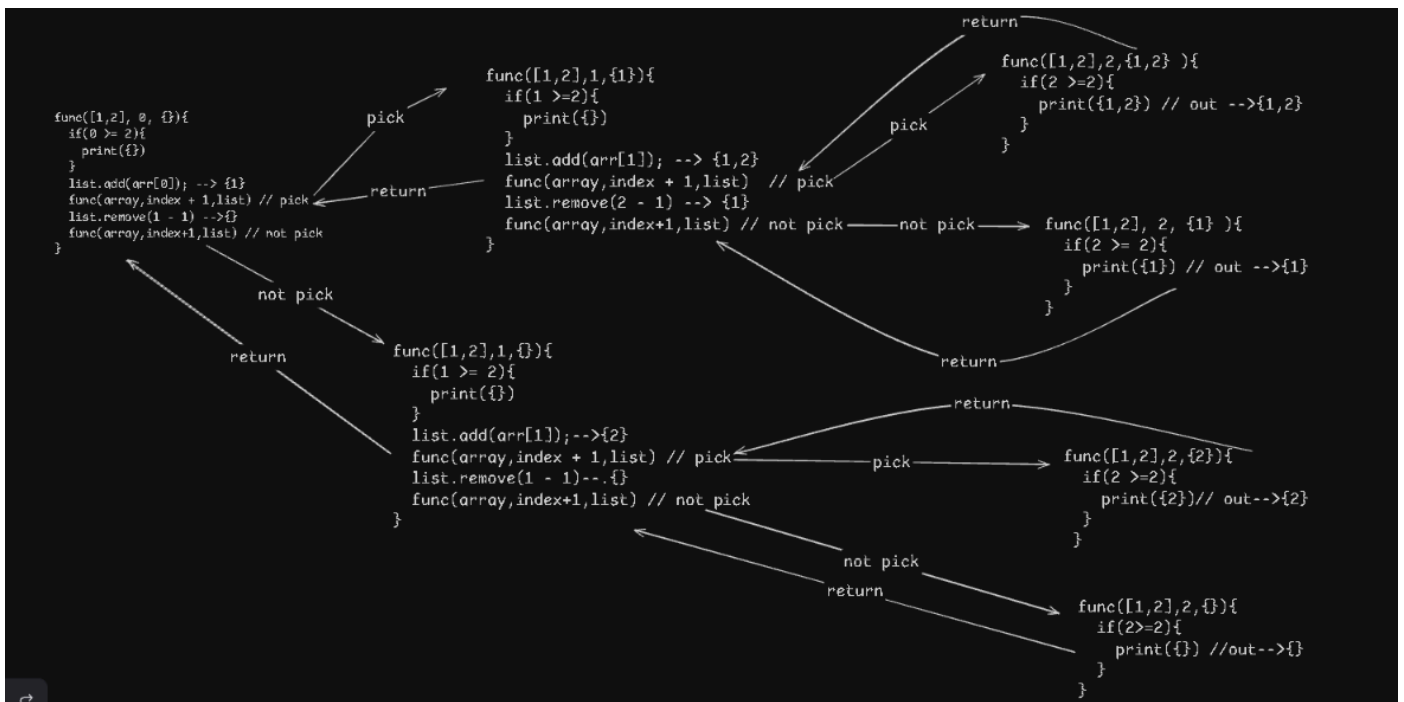
this pattern is really important for solving many other recursion based questions. In this we basically try to print all the subsequence of an array

pseudo code

```
func( array, index, list ){
    if(index >= arr.length){
        print(list)
    }
    list.add(arr[index]);
    func(array, index + 1, list) // pick
    list.remove(list.size() - 1)
    func(array, index+1, list) // not pick
}
```

dry run

now we are going to dry run this above code for better understanding, given array as [1,2]



• Subsequence

A subsequence is a contiguous or non contiguous sequence which follows an order

For example --

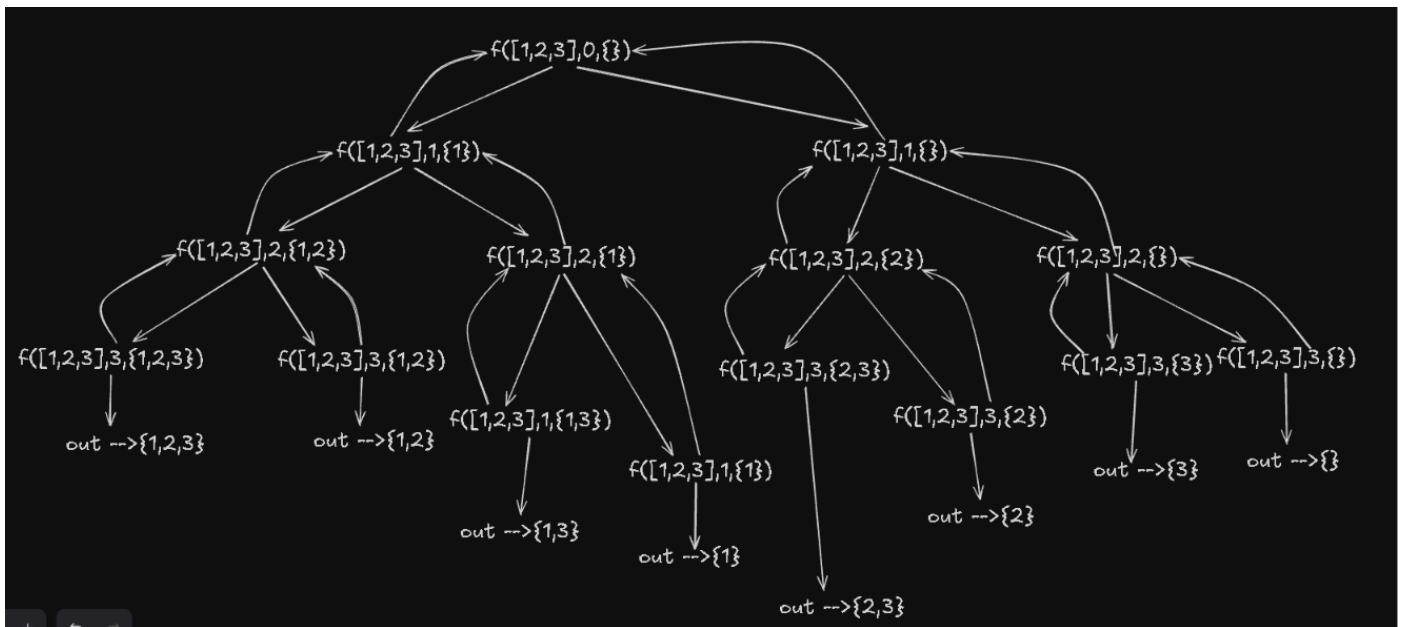
we have an array --> [1,2,3]

its subsequences are --

- []
- [1]
- [3]
- [1,3] and so on ..

but remember here order is important so here [3,2] is not a subsequence but an subarray of the given array.

✓ Recursion Tree for arr = [1,2,3]



Code

```

class Solution {
    public void helper(int [] arr, List<List<Integer>> l1, List<Integer> l2, int index){
        if(index >= arr.length){
            l1.add(new ArrayList<>(l2)); // store copy of l2
            return;
        }
        l2.add(arr[index]);
        helper(arr, l1, l2, index+1);
        l2.remove(l2.size()-1);
        helper(arr, l1, l2, index+1);
    }
    public List<List<Integer>> subsets(int[] nums) {
        List<List<Integer>> mainlist = new ArrayList<>();
        List<Integer> helperlist = new ArrayList<>();
        helper(nums, mainlist, helperlist, 0);
        return mainlist;
    }
}

```

time complexity = $O(n * 2^n)$

space complexity = $O(n * 2^n)$

